Studio Nx

S1-10a 타임라인 클립 · 연결선 · 패턴 API — 함수 설명 합본 (백엔드 274~281)

2026-10-19

목차

1. ai-notes/backend(fastAPI_Python)/S1-10a - 클립·연결·패턴 API/274_models_clip.md
2. ai-notes/backend(fastAPI_Python)/S1-10a - 클립·연결·패턴 API/275_models_track_connection.md
3. ai-notes/backend(fastAPI_Python)/S1-10a - 클립·연결·패턴 API/276_schemas_clip_connection.md
4. ai-notes/backend(fastAPI_Python)/S1-10a - 클립·연결·패턴 API/277_services_clip.md
5. ai-notes/backend(fastAPI_Python)/S1-10a - 클립·연결·패턴 API/278_services_connection.md
6. ai-notes/backend(fastAPI_Python)/S1-10a - 클립·연결·패턴 API/279_services_pattern.md
7. ai-notes/backend(fastAPI_Python)/S1-10a - 클립·연결·패턴 API/280_api_clips_connections.md
8. ai-notes/backend(fastAPI_Python)/S1-10a - 클립·연결·패턴 API/281_api_patterns.md

백엔드 274 — 클립 모델 (models/clip.py)

항목	내용
파일	`studionx/backend/app/models/clip.py`
작업	S1-10a 클립·연결·패턴 API: 타임라인 클립 표 모델 (PRD 02 FR-2196)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	이전 273 `sequencer-devices.js`, 다음 275 `TrackConnection` 모델

1. 하는 일

트랙 위에 놓인 클립 한 건을 담는 SQLAlchemy 모델입니다. 모델은 실제 MySQL 표 `clips` 와 1:1 로 맞춰져 있고, `studionx/db/02_phase2_sequencer.sql` 과 `docs/DB.md` §3.3 의 정의와 열 이름이 같습니다.

```
clips 한 행 (트랙 위 클립)
├ clip_type = MIDI  — pattern_id 필수, sample_id 비움. 노트는 패턴(midi_notes)에 있음
├ clip_type = AUDIO — sample_id 필수, pattern_id 비움
├ 위치와 길이       — start_tick, length_tick, offset_tick (1박 = 960 tick)
└ 소리 모양 칸      — name, gain_db, is_looped, fade_in_tick, fade_out_tick,
                     fade_in_curve, fade_out_curve, warp_mode, warp_markers,
                     pitch_semitones, is_reversed
```

트랙을 지우면 DB 가 `ON DELETE CASCADE` 로 클립도 함께 지웁니다. 반대로 패턴과 샘플에는 `ON DELETE` 가 없어서 DB 가 RESTRICT 로 막습니다. 그래서 클립이 가리키는 패턴은 지울 수 없습니다.

쓰는 곳은 `clip_service` 의 목록, 만들기, 수정, 삭제, `api/v1/clips.py`, `full_project_service` 가 채우는 `/projects/{id}/full` 의 `clips` 목록, 그리고 패턴을 지우기 전에 확인하는 `pattern_is_used` 입니다.

2. 코드

```
class ClipType(StrEnum):
    AUDIO = "AUDIO"
    MIDI = "MIDI"


class WarpMode(StrEnum):
    """오디오 클립의 시간 늘리기 방식 (FR-2196). BEATS 는 박자를 맞춰 늘린다."""

    OFF = "OFF"
    BEATS = "BEATS"
    TONES = "TONES"
    TEXTURE = "TEXTURE"
    REPITCH = "REPITCH"
    COMPLEX = "COMPLEX"


class Clip(Base):
    """타임라인 클립 한 건. MIDI 는 pattern_id, AUDIO 는 sample_id 를 가진다."""
```

```
    __tablename__ = "clips"

    clip_id: Mapped[int] = mapped_column(BigInteger, primary_key=True, autoincrement=True)
    track_id: Mapped[int] = mapped_column(
        BigInteger, ForeignKey("tracks.track_id", ondelete="CASCADE")
    )
    clip_type: Mapped[ClipType] = mapped_column(Enum(ClipType))
    sample_id: Mapped[int | None] = mapped_column(BigInteger, ForeignKey("samples.sample_id"))
    pattern_id: Mapped[int | None] = mapped_column(
        BigInteger, ForeignKey("patterns.pattern_id")
    )
    name: Mapped[str | None] = mapped_column(String(100))
    start_tick: Mapped[int] = mapped_column(INTEGER(unsigned=True))
    length_tick: Mapped[int] = mapped_column(INTEGER(unsigned=True))
    offset_tick: Mapped[int] = mapped_column(INTEGER(unsigned=True), server_default=text("0"))
    gain_db: Mapped[Decimal] = mapped_column(Numeric(5, 2), server_default=text("0.00"))
    is_looped: Mapped[bool] = mapped_column(Boolean, server_default=text("0"))
    fade_in_tick: Mapped[int] = mapped_column(INTEGER(unsigned=True), server_default=text("0"))
    fade_out_tick: Mapped[int] = mapped_column(INTEGER(unsigned=True), server_default=text("0"))
    fade_in_curve: Mapped[Decimal] = mapped_column(Numeric(3, 2), server_default=text("0.00"))
    fade_out_curve: Mapped[Decimal] = mapped_column(Numeric(3, 2), server_default=text("0.00"))
    warp_mode: Mapped[WarpMode] = mapped_column(
        Enum(WarpMode), server_default=WarpMode.BEATS
    )
    warp_markers: Mapped[dict[str, Any] | None] = mapped_column(JSON)
    pitch_semitones: Mapped[Decimal] = mapped_column(Numeric(5, 2), server_default=text("0.00"))
    is_reversed: Mapped[bool] = mapped_column(Boolean, server_default=text("0"))
    created_at: Mapped[datetime] = mapped_column(
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)")
    )
    updated_at: Mapped[datetime] = mapped_column(
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)"), server_onupdate=FetchedValue()
    )
```

문법 풀이

식	뜻
`__tablename__ = "clips"`	이 모델이 실제 표 `clips` 를 담당함을 적습니다
`Mapped[int]` 와 `mapped_column(...)`	파이썬 칸에 DB 열의 자료형과 기본값을 함께 적는 최신 표기법입니다
`ForeignKey("tracks.track_id", ondelete="CASCADE")`	트랙을 지우면 DB 가 이 클립도 함께 지웁니다
`ForeignKey("samples.sample_id")` 와 `ForeignKey("patterns.pattern_id")`	`ON DELETE` 가 없어 DB 가 RESTRICT 로 막습니다. 클립이 쓰는 샘플과 패턴은 지울 수 없습니다
`Mapped[int | None]`	비어 있을 수 있는 칸입니다. 종류에 따라 `sample_id` 와 `pattern_id` 중 하나만 값이 들어갑니다
`Enum(ClipType)`	`"AUDIO"` 와 `"MIDI"` 두 글자만 받는 DB ENUM 입니다
`StrEnum`	열거형을 글자 그대로 비교하게 합니다. `clip.clip_type == "MIDI"` 가 참입니다
`server_default=text("0.00")`	값을 안 보내면 DB 가 기본값을 채웁니다

식	뜻
`Numeric(5, 2)`	소수 둘째 자리까지의 숫자입니다. `gain_db` 와 `pitch_semitones` 가 이 형태입니다
`INTEGER(unsigned=True)`	tick 은 음수가 되지 않으므로 0 부터 약 42억까지 씁니다
`server_onupdate=FetchedValue()`	행을 고치면 `updated_at` 을 DB 가 알아서 바꿉니다
`JSON`	`warp_markers` 처럼 모양이 정해지지 않은 값을 그대로 담습니다

3. 왜 이렇게

선택	이유
종류 규칙을 모델이 아니라 DB CHECK 에 둠	`chk_clips_kind` 가 MIDI 는 `pattern_id` 만, AUDIO 는 `sample_id` 만 갖게 합니다. DB 가 마지막으로 지키므로 어떤 코드 경로로도 어긋난 행이 들어가지 않습니다
같은 규칙을 요청 스키마가 먼저 검사	`ClipCreateRequest._kind_matches_ids` 가 422 로 막아 DB 까지 가지 않습니다 (276 문서)
`track_id` 에만 CASCADE	트랙이 곧 클립의 집입니다. 트랙이 지워지면 거기에 놓인 클립도 쓸모가 없습니다
`pattern_id` 와 `sample_id` 에 ON DELETE 를 안 씀	패턴과 샘플을 지우면 노트와 소리가 통째로 사라집니다. 클립이 없는 패턴과 쓰이지 않는 샘플만 지울 수 있게 RESTRICT 로 막았습니다
fade 칸과 warp 칸을 지금부터 넣음	Phase 2 재생 확장용입니다. 나중에 표를 바꾸지 않도록 칸과 기본값을 미리 맞춰 둡니다
`created_at` 과 `updated_at` 을 `DATETIME(fsp=3)` 로	DB 표 정의와 같고, 밀리초까지 기록합니다
클래스에 `__table_args__` 없음	CHECK 와 인덱스는 SQL 쪽(`02_phase2_sequencer.sql`)에 이미 있으므로 모델에 다시 쓰지 않습니다

4. 검증

`tests/test_clip_model.py` 다섯 건이 모두 통과합니다. `test_clip_columns_match_db` 는 칸 이름, 순서, NULL 허용, FK 를 실제 MySQL 표와 대조하고(assert_matches_db), `test_midi_clip_defaults` 와 `test_audio_clip_defaults` 는 종류별 기본값, `test_deleting_track_deletes_clips` 는 CASCADE, `test_pattern_used_by_clip_cannot_be_deleted` 는 RESTRICT 로 인한 `IntegrityError` 를 확인합니다. `tests/test_pattern_routes.py` 의 `test_delete_pattern_used_by_clip_is_400` 은 클립이 쓰는 패턴 삭제가 400 으로 막히는지, `tests/test_full_project_routes.py` 의 `test_full_includes_clips_and_connections` 은 `/full` 응답에 클립이 담기는지 봅니다. `pytest -q` 는 1083 passed, `ruff check .` 는 오류 없이 통과합니다.

5. 남은 일 (사용자)

확인할 것	내용
문서 검토	S1-10a 함수 문서(274 부터 276 까지)가 모두 끝난 뒤 한 번에 검토합니다
DB 변경 없음	`clips` 표는 이미 로컬 MySQL에 있어 마이그레이션을 따로 실행하지 않습니다

백엔드 275 — 트랙 연결 모델 (models/track_connection.py)

항목	내용
파일	`studionx/backend/app/models/track_connection.py`
작업	S1-10a 클립·연결·패턴 API: 트랙 간 연결 표 모델 (PRD 02 FR-2197)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	이전 274 `Clip` 모델, 다음 276 `schemas/clip.py` 와 `schemas/connection.py`

1. 하는 일

화면의 Connect to Another 칸이 저장하는 연결 한 건을 담는 SQLAlchemy 모델입니다. 모델은 실제 MySQL 표 `track_connections` 와 1:1 로 맞춰져 있고, `studionx/db/02_phase2_sequencer.sql` 과 `docs/DB.md` §3.2 의 정의와 열 이름이 같습니다.

```
track_connections 한 행 (보내는 트랙마다 최대 하나, DB UNIQUE uq_connections_from)
├ from_track_id       – 보내는 트랙
├ to_track_id         – 받는 트랙
├ target              – PLUGIN(이펙트 입력) 또는 INSTRUMENTS(악기 입력), 기본 PLUGIN
├ to_track_plugin_id  – target 이 PLUGIN 일 때 받는 트랙의 이펙트 한 칸
├ to_port             – 받는 트랙의 입력 번호, 기본 1
├ also_play           – NULL 이면 자동, true/false 면 켜짐, 꺼짐을 직접 정함
└ volume_db, pan, is_solo, created_at
```

보내는 트랙 하나에 연결은 하나뿐입니다. DB 의 `UNIQUE uq_connections_from` 가 이를 막고, 그래서 저장은 새로 만드는 것이 아니라 보낸 값으로 통째로 바꾸는 방식으로 다룹니다. 보내는 트랙이나 받는 트랙을 지우면 DB 가 `ON DELETE CASCADE` 로 연결도 함께 지웁니다.

쓰는 곳은 `connection_service` 의 조회, 통째로 맞추기, 끄기, `api/v1/connections.py`, 그리고 `full_project_service` 가 트랙마다 채우는 `/projects/{id}/full` 의 `connections` 목록입니다.

2. 코드

```
class ConnectionTarget(StrEnum):
    """받는 쪽에서 무엇으로 받을지. PLUGIN 이면 이펙트 입력, INSTRUMENTS 면 악기 입력이다."""

    PLUGIN = "PLUGIN"
    INSTRUMENTS = "INSTRUMENTS"


class TrackConnection(Base):
    """트랙 → 트랙 연결 한 건. 보내는 트랙마다 최대 하나다 (UNIQUE from_track_id)."""

    __tablename__ = "track_connections"

    connection_id: Mapped[int] = mapped_column(BigInteger, primary_key=True, autoincrement=True)
```

```
    from_track_id: Mapped[int] = mapped_column(
        BigInteger, ForeignKey("tracks.track_id", ondelete="CASCADE")
    )
    to_track_id: Mapped[int] = mapped_column(
        BigInteger, ForeignKey("tracks.track_id", ondelete="CASCADE")
    )
    target: Mapped[ConnectionTarget] = mapped_column(
        Enum(ConnectionTarget), server_default=ConnectionTarget.PLUGIN
    )
    to_track_plugin_id: Mapped[int | None] = mapped_column(
        BigInteger, ForeignKey("track_plugins.track_plugin_id", ondelete="CASCADE")
    )
    to_port: Mapped[int] = mapped_column(TINYINT(unsigned=True), server_default=text("1"))
    also_play: Mapped[bool | None] = mapped_column(Boolean)
    volume_db: Mapped[Decimal] = mapped_column(Numeric(5, 2), server_default=text("0.00"))
    pan: Mapped[Decimal] = mapped_column(Numeric(3, 2), server_default=text("0.00"))
    is_solo: Mapped[bool] = mapped_column(Boolean, server_default=text("0"))
    created_at: Mapped[datetime] = mapped_column(
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)")
    )
```

문법 풀이

식	뜻
`__tablename__ = "track_connections"`	이 모델이 실제 표 `track_connections` 를 담당함을 적습니다
`ForeignKey("tracks.track_id", ondelete="CASCADE")` (`from_track_id` 와 `to_track_id`)	보내는 트랙이나 받는 트랙이 지워지면 연결도 함께 지워집니다
`ForeignKey("track_plugins.track_plugin_id", ondelete="CASCADE")`	이펙트 칸이 지워지면 그 칸을 가리키던 연결도 함께 지워집니다
`Enum(ConnectionTarget), server_default=ConnectionTarget.PLUGIN`	값을 안 보내면 `PLUGIN` 으로 채워집니다
`Mapped[bool | None]`	`also_play` 처럼 세 가지 경우를 담습니다. NULL 은 자동 결정을 뜻합니다
`TINYINT(unsigned=True)`	입력 번호는 0 부터 255 까지의 정수이고, `to_port` 기본값은 1 입니다
`Numeric(5, 2)` 와 `Numeric(3, 2)`	`volume_db` 와 `pan` 을 소수 둘째 자리까지 담습니다
`DATETIME(fsp=3)`	`created_at` 을 밀리초 단위로 적습니다. 이 표에는 `updated_at` 칸이 없습니다

3. 왜 이렇게

선택	이유
UNIQUE 를 SQL 쪽에만 둠	`uq_connections_from` 은 `02_phase2_sequencer.sql` 에 이미 있습니다. 모델에 `__table_args__` 로 다시 쓰면 같은 규칙이 두 곳에서 어긋날 수 있습니다
저장을 PUT 으로 둠	만들기와 고치기를 나누면 두 행을 넣는 경로가 생깁니다. 보낸 값으로 통째로 맞추면 늘 행 하나여서 UNIQUE와 맞습니다 (FR-2197)
양쪽 트랙 FK 모두 CASCADE	받는 트랙을 지우면 연결이 자연스럽게 사라집니다. 화면이 보는 연결이 None 으로 돌아가는 것과 코드 한 줄 없이 일치합니다

선택	이유
`also_play` 를 NULL 허용	NULL 이면 받는 쪽이 소리도 내는지를 자동으로 정하고, 켜짐 또는 꺼짐을 직접 고르면 true/false 가 들어갑니다
`target` 기본값을 PLUGIN 으로	화면에서 Connect 를 켜면 이펙트 입력으로 먼저 연결되는 동작과 같습니다
자기 자신, 서로, 고리 검사를 스키마에 안 씀	두 트랙 사이의 관계는 표 한 줄로는 알 수 없습니다. `connection_service` 가 400 으로 막습니다
`created_at` 만 있고 `updated_at` 없음	DB 표 정의 그대로입니다. 연결을 고쳐도 만든 시각만 남습니다

4. 검증

`tests/test_track_connection_model.py` 네 건이 모두 통과합니다. `test_connection_columns_match_db` 는 칸 이름, 순서, NULL 허용, FK 를 실제 MySQL 표와 대조하고(assert_matches_db), `test_connection_defaults` 는 기본값(`target` 이 `PLUGIN`, `to_port` 가 1, `also_play` 가 NULL)을, `test_deleting_source_track_deletes_connection` 은 보내는 트랙을 지울 때의 CASCADE, `test_deleting_target_track_deletes_connection` 은 받는 트랙을 지울 때의 CASCADE 를 확인합니다. `tests/test_full_project_routes.py` 의 `test_full_includes_clips_and_connections` 와 `tests/test_connection_routes.py` 도 함께 통과합니다. `pytest -q` 는 1083 passed, `ruff check .` 는 오류 없이 통과합니다.

5. 남은 일 (사용자)

확인할 것	내용
문서 검토	S1-10a 함수 문서(274 부터 276 까지)가 모두 끝난 뒤 한 번에 검토합니다
DB 변경 없음	`track_connections` 표는 이미 로컬 MySQL에 있어 마이그레이션을 따로 실행하지 않습니다

백엔드 276 — 클립 · 연결 스키마 (schemas/clip.py · schemas/connection.py)

항목	내용
파일	`studionx/backend/app/schemas/clip.py`, `studionx/backend/app/schemas/connection.py`
작업	S1-10a 클립·연결·패턴 API: 클립과 연결의 요청과 응답 스키마 (PRD 02 FR-2196, FR-2197)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	이전 275 `TrackConnection` 모델, 다음 S1-10a 의 서비스와 라우터 문서

1. 하는 일

클립 API 와 연결 API 가 주고받는 요청과 응답의 모양을 정합니다. 요청 칸의 범위를 검사해 잘못된 값이 DB 에 닿기 전에 422 로 막고, 응답은 DB 행을 그대로 되돌려 줍니다.

```
POST   /tracks/{track_id}/clips       ClipCreateRequest         → ClipResponse (201)
PATCH  /clips/{clip_id}               ClipUpdateRequest         → ClipResponse
PUT    /tracks/{track_id}/connection  TrackConnectionSetRequest → TrackConnectionResponse
GET    /projects/{id}/full            한 번의 조회              → clips 목록, 트랙별 connections
```

주요 검사는 다음과 같습니다.

칸	검사	뜻
`name`	앞뒤 공백을 지우고 1자 이상 100자 이하	DB `VARCHAR(100)` 과 같은 길이입니다
`start_tick`	0 이상	타임라인 시작 위치라 음수가 될 수 없습니다
`length_tick`	1 이상	길이가 0인 클립은 없으므로 0 은 422 로 거절됩니다
`offset_tick`, `fade_in_tick`, `fade_out_tick`	0 이상	트림 위치와 페이드 길이는 음수가 아닙니다
`gain_db`, `volume_db`	-60 이상 24 이하	허용하는 dB 범위입니다
`pan`	-1 이상 1 이하	왼쪽 끝부터 오른쪽 끝까지입니다
`fade_in_curve`, `fade_out_curve`	-1 이상 1 이하	DB 의 `chk_clips_fade_curve` 와 같습니다
`pitch_semitones`	-48 이상 48 이하	DB 의 `chk_clips_pitch` 와 같습니다
`pattern_id`, `sample_id`, `to_track_id`, `to_track_plugin_id`	1 이상	번호 칸이라 0 또는 음수를 받지 않습니다
`to_port`	1 이상 16 이하	받는 트랙의 입력 번호 범위입니다
`clip_type`, `target`, `warp_mode`	`Literal` 의 값만	DB ENUM 과 같은 글자만 통과합니다

2. 코드

요청 (`clip.py`)

```
class ClipCreateRequest(BaseModel):
    """POST /tracks/{id}/clips — 타임라인에 클립을 놓는다.

    MIDI 클립은 pattern_id 를, AUDIO 클립은 sample_id 를 반드시 보낸다 (반대쪽은 비운다).
    """

    clip_type: ClipType
    pattern_id: int | None = Field(default=None, ge=1)
    sample_id: int | None = Field(default=None, ge=1)
    name: ClipName | None = None
    start_tick: int = Field(ge=0)
    length_tick: int = Field(ge=1)
    offset_tick: int = Field(default=0, ge=0)
    gain_db: Decimal = Field(default=Decimal("0.00"), ge=-60, le=24)
    is_looped: bool = False
    fade_in_tick: int = Field(default=0, ge=0)
    fade_out_tick: int = Field(default=0, ge=0)
    fade_in_curve: Decimal = Field(default=Decimal("0.00"), ge=-1, le=1)
    fade_out_curve: Decimal = Field(default=Decimal("0.00"), ge=-1, le=1)
    warp_mode: WarpMode = "BEATS"
    warp_markers: dict[str, Any] | None = None
    pitch_semitones: Decimal = Field(default=Decimal("0.00"), ge=-48, le=48)
    is_reversed: bool = False

    @model_validator(mode="after")
    def _kind_matches_ids(self) -> Self:
        """MIDI 는 pattern_id 만, AUDIO 는 sample_id 만 — DB CHECK 와 같은 규칙."""
        if self.clip_type == "MIDI":
            if self.pattern_id is None or self.sample_id is not None:
                raise ValueError("MIDI clip needs pattern_id only")
        elif self.sample_id is None or self.pattern_id is not None:
            raise ValueError("AUDIO clip needs sample_id only")
        return self
```

요청 (`connection.py`)

```
class TrackConnectionSetRequest(BaseModel):
    """PUT /tracks/{id}/connection — 이 트랙에서 나가는 연결을 통째로 맞춘다.

    to_track_id 는 받는 트랙이다. target 이 PLUGIN 이면 to_track_plugin_id 로 그 트랙의
    이펙트 한 칸을 고른다(없으면 트랙 입력). INSTRUMENTS 면 악기 입력이라 이펙트는 없다.
    """

    to_track_id: int = Field(ge=1)
    target: ConnectionTarget = "PLUGIN"
    to_track_plugin_id: int | None = Field(default=None, ge=1)
    to_port: int = Field(default=1, ge=1, le=16)
    also_play: bool | None = None
    volume_db: Decimal = Field(default=Decimal("0.00"), ge=-60, le=24)
    pan: Decimal = Field(default=Decimal("0.00"), ge=-1, le=1)
    is_solo: bool = False
```

나머지 세 클래스는 칸만 봐도 알 수 있습니다. `ClipUpdateRequest` 는 모든 칸이 `| None` 이라서 `clip_service` 가 `exclude_unset=True` 로 보낸 칸만 고치고, 종류와 `pattern_id`, `sample_id` 는 아예 넣지 않았습니다. 다른 종류의 클립으로 바꾸는 일은 없기 때문입니다. `ClipResponse` 는 `ConfigDict(from_attributes=True)` 로 DB 행을 그대로 받아 `clip_id` 부터 `updated_at` 까지 20개 칸을 돌려주고, `created_at` 은 제외합니다. `TrackConnectionResponse` 는 같은 방식으로 `connection_id` 부터 `is_solo` 까지 10개 칸을 돌려줍니다.

문법 풀이

식	뜻
`ClipType = Literal["AUDIO", "MIDI"]`	`AUDIO` 와 `MIDI` 두 글자만 허용하고, DB ENUM 과 같은 값을 씁니다
`ClipName = Annotated[str, StringConstraints(strip_whitespace=True, min_length=1, max_length=100)]`	이름의 앞뒤 공백을 지우고 1자 이상 100자 이하로 잡습니다
`CLIP_NAME_MAX = 100`	`clips.name` 칸 길이(`VARCHAR(100)`)와 맞춘 상수입니다
`Field(ge=0)`, `Field(ge=1)`, `Field(ge=-60, le=24)`	값이 몇 이상, 몇 이하인지 적습니다. 범위를 벗어나면 422 가 납니다
`@model_validator(mode="after")`	칸을 다 채운 뒤 한 번 더 검사합니다. 실패하면 FastAPI 가 422 로 바꿉니다
`raise ValueError("MIDI clip needs pattern_id only")`	규칙에 어긋난 요청을 막는 메시지입니다
`int | None = Field(default=None, ge=1)`	안 보내면 없음이고, 보내면 1 이상인 번호입니다
`warp_markers: dict[str, Any] | None`	모양이 정해지지 않은 값을 그대로 주고받습니다
`ConfigDict(from_attributes=True)`	ORM 행을 응답으로 그대로 바꿀 수 있게 합니다

3. 왜 이렇게

선택	이유
종류별 규칙을 `model_validator` 로 검사	필드 하나로는 MIDI 면 `pattern_id` 필수 같은 짝을 표현할 수 없습니다. 클래스를 다 만든 뒤 한 번 보고, DB 의 `chk_clips_kind` 와 같은 규칙을 422 로 막습니다
수정 요청의 모든 칸을 `None` 기본값으로	보낸 칸만 바꾸는 PATCH 와 맞습니다
종류와 `pattern_id`, `sample_id` 를 수정 요청에서 제외	다른 패턴이나 샘플로 바꾸는 일은 없습니다. 바꾸려면 새 클립을 만듭니다
응답에 `created_at` 를 빼고 `updated_at` 만 담음	화면이 필요로 하는 것은 마지막 수정 시각뿐입니다. `tests/test_clip_routes.py` 의 `CLIP_KEYS` 20개 칸과 맞습니다
연결 저장을 POST 와 DELETE 가 아니라 PUT 으로	보내는 트랙마다 행이 하나뿐이라(DB `uq_connections_from`) 통째로 맞추는 편이 표 규칙과 맞습니다
`to_track_id` 와 `to_track_plugin_id` 를 나눔	`target` 이 INSTRUMENTS 면 이펙트 칸이 필요 없습니다. 서비스가 INSTRUMENTS 일 때 이펙트 칸을 비웁니다
자기 자신, 서로, 고리 검사는 스키마에서 제외	두 트랙 사이의 관계는 표 한 줄로 알 수 없습니다. `connection_service` 가 400 으로 막습니다

4. 검증

`tests/test_clip_routes.py` 에서 `test_create_clip_validates_kind` 가 MIDI 인데 `pattern_id` 가 없는 경우, `sample_id` 까지 갖춘 경우, `length_tick` 이 0인 경우를 모두 422 로 확인하고, 응답 칸은 `CLIP_KEYS` 20개로 검사합니다. `tests/test_connection_routes.py` 에서 `test_put_creates_and_get_returns` 와 `test_put_replaces_existing` 은 PUT 통째로 맞추기, `test_put_instruments_target_drops_plugin` 은 INSTRUMENTS 일 때 이펙트 칸이 비워지는 것, `test_put_rejects_self_connection`, `test_put_rejects_mutual_connection`, `test_put_rejects_loop` 은 잘못된 연결의 400 을 확인합니다. `pytest -q` 는 1083 passed, `ruff check .` 는 오류 없이 통과합니다.

5. 남은 일 (사용자)

확인할 것	내용
문서 검토	274 와 275 문서와 함께 S1-10a 작업 단위가 끝난 뒤 한 번에 검토합니다
`/docs` 에서 확인	`POST /tracks/{id}/clips` 에 `length_tick: 0` 을 넣으면 422 가 나는지 눌러 볼 수 있습니다

백엔드 277 — 클립 서비스 (services/clip_service.py)

항목	내용
파일	`studionx/backend/app/services/clip_service.py`
작업	S1-10a 클립·연결·패턴 API: 타임라인 클립의 목록, 만들기, 수정, 삭제 서비스 (PRD 02 FR-2196)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	이전 276 클립·연결 스키마, 다음 278 연결 서비스

1. 하는 일

트랙에 놓인 클립 한 건을 읽고 쓰는 일을 모두 담당합니다. 권한과 404 규칙을 이 파일 한 곳에 모아 두어서, 라우터(280)는 요청을 그대로 넘기기만 하면 됩니다.

```
list_clips(db, user, track_id)          트랙 확인 → 클립 목록을 start_tick 순으로 반환
create_clip(db, user, track_id, data)   트랙 권한 확인 → MIDI 는 패턴, AUDIO 는 샘플 확인 → 삽입
update_clip(db, user, clip_id, data)    클립 찾기 → 트랙 권한 확인 → 보낸 칸만 바꾼 뒤 반환
delete_clip(db, user, clip_id)          클립 찾기 → 트랙 권한 확인 → 삭제하고 패턴은 남김
pattern_is_used(db, pattern_id)         clips 표에서 pattern_id 검색 → 한 건이라도 있으면 True
```

만들기 조건은 다음과 같습니다. MIDI 클립의 패턴은 같은 프로젝트의 것이어야 하며, 없거나 남의 프로젝트 것이면 404로 막습니다. AUDIO 클립의 샘플은 존재해야 하며, 없으면 404로 막습니다. 종류와 가리키는 칸이 어긋나는 요청은 요청 스키마(276)가 422로 먼저 막습니다. 조회는 멤버면 누구나 되고, 쓰기는 OWNER와 EDITOR만 할 수 있습니다.

2. 코드

```
def _require_editor_track(db: Session, user: User, track_id: int) -> Track:
    """쓰기용 트랙 확인 — 남의 트랙은 404, 권한이 없으면 403."""
    track = track_service.require_track(db, user, track_id)
    _project, member = project_service.get_membership(db, track.project_id, user)
    project_service.require_role(member, MemberRole.OWNER, MemberRole.EDITOR)
    return track


def _require_clip(db: Session, user: User, clip_id: int) -> Clip:
    """클립 한 건. 없거나 남의 것이면 404 (권한은 트랙 쪽에서 다시 본다)."""
    clip = db.get(Clip, clip_id)
    if clip is None:
        raise AppError(ErrorCode.NOT_FOUND, 404, "Clip not found.")
    track = track_service.require_track(db, user, clip.track_id)
    if track.track_id != clip.track_id:  # 방어: require_track 이 다른 트랙을 돌려줄 일은 없음
        raise AppError(ErrorCode.NOT_FOUND, 404, "Clip not found.")
    return clip
```

`create_clip` 은 만들기, `delete_clip` 은 삭제, `pattern_is_used` 은 패턴 삭제 전 확인을 맡습니다.

```
def create_clip(
    db: Session, user: User, track_id: int, data: ClipCreateRequest
) -> Clip:
    """트랙에 클립을 놓는다 (FR-2196). OWNER·EDITOR 만.

    MIDI 클립의 패턴은 같은 프로젝트의 것이어야 하고(404), AUDIO 클립의 샘플은
    있어야 한다(404). 종류와 가리키는 칸이 어긋나는 것은 요청 스키마가 422 로 막는다.
    """
    track = _require_editor_track(db, user, track_id)

    if data.clip_type == ClipType.MIDI:
        pattern = db.get(Pattern, data.pattern_id)
        if pattern is None or pattern.project_id != track.project_id:
            raise AppError(ErrorCode.NOT_FOUND, 404, "Pattern not found.")
    else:
        sample = db.get(Sample, data.sample_id)
        if sample is None:
            raise AppError(ErrorCode.NOT_FOUND, 404, "Sample not found.")

    clip = Clip(
        track_id=track.track_id,
        clip_type=data.clip_type,
        sample_id=data.sample_id,
        pattern_id=data.pattern_id,
        name=data.name,
        start_tick=data.start_tick,
        length_tick=data.length_tick,
        offset_tick=data.offset_tick,
        gain_db=data.gain_db,
        is_looped=data.is_looped,
        fade_in_tick=data.fade_in_tick,
        fade_out_tick=data.fade_out_tick,
        fade_in_curve=data.fade_in_curve,
        fade_out_curve=data.fade_out_curve,
        warp_mode=data.warp_mode,
        warp_markers=data.warp_markers,
        pitch_semitones=data.pitch_semitones,
        is_reversed=data.is_reversed,
    )
    db.add(clip)
    db.commit()
    db.refresh(clip)
    return clip


def delete_clip(db: Session, user: User, clip_id: int) -> None:
    """클립 삭제 (FR-2196). OWNER·EDITOR 만. 클립이 가리키던 패턴은 그대로 남는다."""
    clip = _require_clip(db, user, clip_id)
    _require_editor_track(db, user, clip.track_id)
    db.delete(clip)
    db.commit()


def pattern_is_used(db: Session, pattern_id: int) -> bool:
    """이 패턴을 가리키는 클립이 있는지 — 패턴 삭제 전에 본다 (FR-2196)."""
    return (
```

```
        db.scalar(select(Clip.clip_id).where(Clip.pattern_id == pattern_id).limit(1))
        is not None
    )
```

문법 풀이

식	뜻
`track_service.require_track(db, user, track_id)`	트랙이 없거나 남의 프로젝트 것이면 404로 막습니다
`project_service.require_role(member, MemberRole.OWNER, MemberRole.EDITOR)`	VIEWER가 쓰기를 시도하면 403으로 막습니다
`db.get(Pattern, data.pattern_id)`	패턴 한 건을 읽고, 없으면 404로 막습니다
`pattern.project_id != track.project_id`	남의 프로젝트 패턴을 가리키는 클립은 404로 막습니다
`db.add(clip)`	클립 한 행을 세션에 올립니다
`db.commit()`	지금까지의 변경을 DB에 한 번에 확정합니다
`db.refresh(clip)`	확정 직후 DB에 적힌 최신 값(번호, 시각)을 다시 읽습니다
`data.model_dump(exclude_unset=True)`	요청에 실제로 담긴 칸만 골라서 사전으로 만듭니다
`setattr(clip, field, value)`	골라낸 칸 이름으로 클립의 값을 바꿉니다. 보낸 칸만 바뀝니다
`select(Clip.clip_id).where(...).limit(1)`	행을 채우지 않게 번호 한 개만 찾아 있고 있는지 봅니다

3. 왜 이렇게

선택	이유
쓰기 권한을 `_require_editor_track` 한 곳에 모음	만들기, 수정, 삭제가 같은 규칙을 씁니다. 남의 트랙은 404, 권한 없음은 403입니다
클립 존재와 트랙 권한을 `_require_clip` 이 함께 확인	클립 번호로 먼저 찾은 뒤 그 클립의 트랙에 대해 권한을 다시 확인합니다
수정은 보낸 칸만 바꿈	PATCH 요청은 부분 수정이라 보내지 않은 칸은 그대로 둡니다
삭제가 패턴을 함께 지우지 않음	패턴은 여러 클립이 쓸 수 있고, 지우면 노트가 통째로 사라집니다
`pattern_is_used` 를 클립 쪽에 둠	패턴을 쓰는 클립이 있는지 묻는 질문이라 클립 표를 가장 잘 아는 이 파일이 답합니다
목록을 `start_tick` 순으로 정렬	타임라인을 왼쪽부터 그리는 화면의 순서와 같습니다

4. 검증

`tests/test_clip_routes.py` 열한 건이 모두 통과합니다. `test_create_midi_clip_201` 과 `test_create_audio_clip_201` 은 종류별 만들기 201을, `test_create_clip_to_other_project_pattern_is_404` 는 남의 프로젝트 패턴 404를, `test_create_clip_nonexistent_sample_is_404` 는 없는 샘플 404를 확인합니다. `test_patch_clip` 은 보낸 칸만 바뀌는지, `test_delete_clip` 은 204인지를, `test_viewer_can_read_but_not_write` 와 `test_stranger_gets_404` 와 `test_needs_login` 은 권한과 로그인 규칙을 확인합니다.

`tests/test_full_project_routes.py` 의 `test_full_includes_clips_and_connections` 는 전체 조회에 클립이 담기는지 봅니다. `pytest -q` 는 1083 passed, `ruff check .` 는 오류 없이 통과합니다.

5. 남은 일 (사용자)

확인할 것	내용
문서 검토	S1-10a 서비스와 라우터 문서(277 부터 281 까지)가 모두 끝난 뒤 한 번에 검토합니다
API 실제 호출 확인	화면에서 클립을 놓고 옮기고 지우는 동작을 시퀀서에서 직접 확인합니다

백엔드 278 — 연결 서비스 (services/connection_service.py)

항목	내용
파일	`studionx/backend/app/services/connection_service.py`
작업	S1-10a 클립·연결·패턴 API: 트랙 간 연결(Connect to Another)의 읽기, 저장, 끄기 서비스 (PRD 02 FR-2197)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	이전 277 클립 서비스, 다음 279 패턴 서비스

1. 하는 일

한 트랙에서 다른 트랙으로 나가는 연결 한 건을 관리합니다. DB에 `uq_connections_from` 유니크가 있어서 `from_track_id` 하나에는 연결이 하나씩만 들어갑니다. 그래서 저장은 "보낸 값으로 통째로 맞추기", 지우기는 "연결 끄기" 두 가지뿐입니다.

```
get_connection(db, user, track_id)        트랙 확인 → 나가는 연결 조회 → 없으면 404
set_connection(db, user, track_id, data)  트랙 권한 확인 → 네 가지 검사 → 없으면 만들고 있으면 통째로 덮어씀
delete_connection(db, user, track_id)     트랙 권한 확인 → 나가는 연결 삭제 → 없으면 그냥 지나감
```

저장 전에 하는 검사는 네 가지입니다. 자기 자신으로는 보낼 수 없으므로 400으로 막고, 받는 트랙이 없는 트랙이거나 다른 프로젝트 것이면 404로 막습니다. 두 트랙이 서로를 가리키면 400으로 막고, 연결을 따라가다 다시 이 트랙에 닿는 고리가 생기면 400으로 막습니다. `target` 이 PLUGIN이면 받는 트랙의 이펙트 한 칸을 가리키어야 하며, 아니면 404로 막습니다.

2. 코드

```
def _outgoing(db: Session, from_track_id: int) -> TrackConnection | None:
    """이 트랙에서 나가는 연결 한 건 (없으면 None)."""
    return db.scalar(
        select(TrackConnection).where(TrackConnection.from_track_id == from_track_id)
    )


def _reaches(db: Session, start: int, target: int) -> bool:
    """start 에서 나가는 연결을 따라가다 target 에 닿는지 본다 (고리 검사).

    트랙마다 나가는 연결이 하나뿐이라 단순히 한 줄로 따라가면 된다. 지나간 트랙을
    다시 만나면 이미 고리이므로 멈춘다 (무한 반복 방지).
    """
    seen: set[int] = set()
    current: int | None = start
    while current is not None and current not in seen:
        if current == target:
            return True
        seen.add(current)
```

```
        nxt = _outgoing(db, current)
        current = nxt.to_track_id if nxt is not None else None
    return current == target
```

`set_connection` 은 저장의 본체이고, `delete_connection` 은 연결을 끕니다.

```
def set_connection(
    db: Session, user: User, track_id: int, data: TrackConnectionSetRequest
) -> TrackConnection:
    """연결을 보낸 값으로 통째로 맞춘다 (FR-2197). 새로 만들거나 지금 것을 고친다.

    잘못된 연결(자기 자신 · 서로 연결 · 고리 · 다른 프로젝트)은 400, 없는 받는 트랙이나
    이펙트는 404 로 막는다.
    """
    track = _require_editor_track(db, user, track_id)

    if data.to_track_id == track.track_id:
        raise AppError(ErrorCode.BAD_REQUEST, 400, "A track cannot connect to itself.")
    to_track = db.get(Track, data.to_track_id)
    if to_track is None or to_track.project_id != track.project_id:
        raise AppError(ErrorCode.NOT_FOUND, 404, "Target track not found.")

    back = _outgoing(db, to_track.track_id)
    if back is not None and back.to_track_id == track.track_id:
        raise AppError(
            ErrorCode.BAD_REQUEST, 400, "Two tracks cannot be connected to each other."
        )
    # 받는 트랙에서 따라가다 이 트랙에 닿으면 고리가 된다. 지금 이 트랙에서 나가는
    # 연결은 곧 바뀌므로, 검사는 받는 트랙 쪽에서만 한다.
    if _reaches(db, to_track.track_id, track.track_id):
        raise AppError(ErrorCode.BAD_REQUEST, 400, "This connection would make a loop.")

    to_track_plugin_id = None
    if data.target == ConnectionTarget.PLUGIN and data.to_track_plugin_id is not None:
        plugin = db.get(TrackPlugin, data.to_track_plugin_id)
        if plugin is None or plugin.track_id != to_track.track_id:
            raise AppError(ErrorCode.NOT_FOUND, 404, "Target effect not found.")
        to_track_plugin_id = plugin.track_plugin_id

    connection = _outgoing(db, track.track_id)
    if connection is None:
        connection = TrackConnection(from_track_id=track.track_id)
        db.add(connection)
    connection.to_track_id = to_track.track_id
    connection.target = data.target
    connection.to_track_plugin_id = to_track_plugin_id
    connection.to_port = data.to_port
    connection.also_play = data.also_play
    connection.volume_db = data.volume_db
    connection.pan = data.pan
    connection.is_solo = data.is_solo
    db.commit()
    db.refresh(connection)
    return connection
```

```
def delete_connection(db: Session, user: User, track_id: int) -> None:
    """연결 끄기 (FR-2197). OWNER·EDITOR 만. 연결이 없어도 그냥 지나간다."""
    track = _require_editor_track(db, user, track_id)
    connection = _outgoing(db, track.track_id)
    if connection is not None:
        db.delete(connection)
        db.commit()
```

문법 풀이

식	뜻
`db.scalar(select(...))`	행이 하나뿐인 조회입니다. 있으면 그 행, 없으면 `None` 을 돌려줍니다
`_outgoing(db, track_id)`	유니크 때문에 이 트랙의 연결은 0개 또는 1개라 한 건으로 족합니다
`seen: set[int]` 와 `while current not in seen`	지나간 트랙 번호를 모아 두어 고리일 때 멈춥니다
`data.to_track_id == track.track_id`	자기 자신으로의 연결은 400으로 막습니다
`to_track.project_id != track.project_id`	남의 프로젝트 트랙은 404로 막습니다
`_reaches(db, to_track_id, from_track_id)`	받는 쪽에서 이 트랙까지 이어지는지 봅니다
`if connection is None:`	같은 트랙의 연결이 있으면 새 행을 만들지 않고 그 행을 고칩니다
`db.add(connection)`	유니크에 걸리지 않는 새 연결 행을 세션에 올립니다
`if connection is not None:`	없는 연결을 지울 때도 오류를 내지 않고 204로 지나갑니다

3. 왜 이렇게

선택	이유
저장을 PUT 으로 둠	한 트랙에서 나가는 연결은 DB 유니크(`uq_connections_from`) 때문에 하나뿐이라, 만들기와 고치기가 결국 같은 동작입니다
DELETE 가 연결을 끄는 역할	화면의 Connect 스위치를 끄는 값과 같습니다. 이미 꺼진 상태라도 204로 조용히 지나갑니다
고리 검사를 받는 트랙 쪽에서 수행	지금 이 트랙에서 나가는 연결은 곧 바뀌므로, 바뀐 뒤에도 고리가 없는지 받는 쪽에서 따라가 봅니다
지나간 트랙을 `seen` 에 기록	연결이 고리일 때 무한 루프에 빠지지 않게 합니다
서로 연결을 따로 검사	A에서 B로, B에서 A로 잇는 짧은 두 연결은 긴 따라가기 검사에서 놓칠 수 있습니다
이펙트는 받는 트랙의 것만 허용	`plugin.track_id != to_track.track_id` 이면 404로 막습니다
권한 확인을 `_require_editor_track` 에 모음	다른 트랙 API와 같은 규칙이라 남의 트랙은 404, 권한 없음은 403입니다

4. 검증

`tests/test_connection_routes.py` 열세 건이 모두 통과합니다. `test_put_creates_and_get_returns` 는 처음 저장과 읽기, `test_put_replaces_existing` 는 두 번째 PUT이 새 연결을 덮어쓰는지 확인합니다.

`test_put_rejects_self_connection`, `test_put_rejects_mutual_connection`, `test_put_rejects_loop` 은 세 가지 잘못된 연결이 400으로 막히는지, `test_put_to_other_project_is_404` 는 남의 프로젝트 404를, `test_delete_turns_connection_off` 는 DELETE 뒤 GET이 404가 되는지를 확인합니다. `test_viewer_cannot_write`,

`test_stranger_gets_404`, `test_needs_login` 은 권한과 로그인 규칙을 확인합니다. `pytest -q` 는 1083 passed, `ruff check .` 는 오류 없이 통과합니다.

5. 남은 일 (사용자)

확인할 것	내용
문서 검토	S1-10a 서비스와 라우터 문서(277 부터 281 까지)가 모두 끝난 뒤 한 번에 검토합니다
화면 연결 확인	Connect to Another 칸에서 트랙과 이펙트를 골라 저장하고 끄는 동작을 시퀀서에서 확인합니다

백엔드 279 — 패턴 서비스 (services/pattern_service.py)

항목	내용
파일	`studionx/backend/app/services/pattern_service.py`
작업	S1-10a 클립·연결·패턴 API: 패턴 만들기, 노트 저장, 패턴 삭제 서비스 (PRD 02 FR-2196, FR-231)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	이전 278 연결 서비스, 다음 280 클립 · 연결 라우터

1. 하는 일

피아노롤 패턴을 만들고, 노트를 저장하고, 쓰이지 않는 패턴을 지우는 일을 담당합니다. 노트는 패턴 하나에 모두 붙어 있어서, 저장할 때마다 그 패턴의 노트를 보낸 목록으로 통째로 바꿉니다.

```
create_pattern(db, user, data)            멤버십 확인 → 권한 확인 → 빈 패턴 삽입 → 노트는 빈 목록으로 201
replace_notes(db, user, pattern_id, data) 패턴과 멤버 확인 → 권한 확인 → 기존 노트 전부 삭제 → 보낸 노트 삽입
delete_pattern(db, user, pattern_id)      패턴과 멤버 확인 → 권한 확인 → 클립 사용 검사 → 쓰지 않는 패턴만 삭제
```

노트 저장은 `PUT /patterns/{id}/notes` 로 받으며, 본문은 노트 목록을 담습니다. 노트 한 건은 `pitch`(0 이상 127 이하), `start_tick`(0 이상), `length_tick`(1 이상), `velocity`(1 이상 127 이하)를 기본으로 하며, `channel`, `release_velocity`, `is_muted`, `probability`, `velocity_deviation` 은 보내지 않으면 기본값이 채워집니다. 보내는 노트는 최대 5000개이고, 빈 목록을 보내면 패턴의 노트가 모두 지워집니다.

2. 코드

```
def _find(db: Session, user: User, pattern_id: int) -> tuple[Pattern, Project, ProjectMember]:
    """패턴과 그 프로젝트·내 멤버 행을 함께 찾는다. 없거나 남의 것이면 404."""
    pattern = db.get(Pattern, pattern_id)
    if pattern is None:
        raise AppError(ErrorCode.NOT_FOUND, 404, "Pattern not found.")
    project, member = project_service.get_membership(db, pattern.project_id, user)
    return pattern, project, member
```

`create_pattern` 은 새 빈 패턴을 만들고, `replace_notes` 가 노트를 통째로 교체합니다.

```
def create_pattern(db: Session, user: User, data: PatternCreateRequest) -> PatternResponse:
    """새 빈 패턴 만들기 (FR-2196). OWNER·EDITOR 만. 노트는 나중에 피아노롤로 채운다."""
    project, member = project_service.get_membership(db, data.project_id, user)
    project_service.require_role(member, MemberRole.OWNER, MemberRole.EDITOR)
    pattern = Pattern(
        project_id=project.project_id,
        name=data.name,
        color=data.color,
        length_tick=data.length_tick,
        swing=data.swing,
    )
```

```
    db.add(pattern)
    db.commit()
    db.refresh(pattern)
    return to_response(pattern, [])


def replace_notes(
    db: Session, user: User, pattern_id: int, data: NotesSaveRequest
) -> PatternResponse:
    """패턴의 노트를 보낸 목록으로 통째 교체한다 (FR-231). OWNER·EDITOR 만.

    (구현: AI — 2026-10-17, S1-08) 지금 붙은 노트를 모두 지우고 보낸 것을 새로 넣는다.
    한 트랜잭션이라 반쪽 상태가 남지 않는다. 빈 목록이면 노트를 모두 지운다.
    """
    pattern, _project, member = _find(db, user, pattern_id)
    project_service.require_role(member, MemberRole.OWNER, MemberRole.EDITOR)

    db.execute(delete(MidiNote).where(MidiNote.pattern_id == pattern.pattern_id))
    db.add_all(
        MidiNote(
            pattern_id=pattern.pattern_id,
            pitch=note.pitch,
            start_tick=note.start_tick,
            length_tick=note.length_tick,
            velocity=note.velocity,
            channel=note.channel,
            release_velocity=note.release_velocity,
            is_muted=note.is_muted,
            probability=note.probability,
            velocity_deviation=note.velocity_deviation,
        )
        for note in data.notes
    )
    db.commit()
    return to_response(pattern, _notes(db, pattern.pattern_id))
```

`delete_pattern` 은 클립이 쓰지 않는 패턴만 지웁니다.

```
def delete_pattern(db: Session, user: User, pattern_id: int) -> None:
    """패턴 삭제 (FR-2196). OWNER·EDITOR 만. 클립이 쓰고 있으면 400 으로 막는다.

    클립이 가리키는 패턴을 지우면 그 클립이 빈 껍데기가 되므로, 마지막 클립을 지운
    화면이 패턴 정리를 이어서 할 수 있게 남은 클립이 없을 때만 지운다.
    """
    pattern, _project, member = _find(db, user, pattern_id)
    project_service.require_role(member, MemberRole.OWNER, MemberRole.EDITOR)
    if clip_service.pattern_is_used(db, pattern.pattern_id):
        raise AppError(ErrorCode.BAD_REQUEST, 400, "Pattern is used by clips.")
    db.delete(pattern)
    db.commit()
```

문법 풀이

식	뜻
`project_service.get_membership(db, project_id, user)`	멤버인지 확인하고, 남의 프로젝트면 있는지조차 알려 주지 않고 404로 막습니다
`project_service.require_role(member, MemberRole.OWNER, MemberRole.EDITOR)`	VIEWER가 저장이나 삭제를 시도하면 403으로 막습니다
`db.execute(delete(MidiNote).where(...))`	이 패턴의 노트를 모두 지웁니다. WHERE 이 없으면 표 전체가 지워집니다
`db.add_all(...)`	지운 자리에 보낸 노트를 한꺼번에 넣습니다
`for note in data.notes`	보낸 목록의 노트를 하나씩 꺼내 `MidiNote` 로 만듭니다
`_notes(db, pattern_id)`	교체가 끝난 뒤 DB 에서 노트를 새로 읽어 옛 목록과 섞이지 않게 합니다
`to_response(pattern, [])`	만들기 직후엔 노트가 없으므로 빈 목록으로 응답합니다
`clip_service.pattern_is_used(db, pattern_id)`	이 패턴을 가리키는 클립이 있으면 참이며, 그러면 400으로 막습니다
`NOTES_MAX = 5000`	한 번에 받는 노트 최대 개수입니다. 더 보내면 422가 납니다

3. 왜 이렇게

선택	이유
노트를 고치지 않고 통째로 교체	피아노롤은 지금 보이는 노트 전부를 다시 보냅니다. 삭제와 삽입을 한 트랜잭션으로 해서 반쪽 상태가 남지 않습니다
교체가 끝나면 `_notes` 로 다시 읽음	세션에 남은 옛 노트 목록과 섞이지 않도록 DB 에서 새로 읽습니다
빈 목록도 받음	노트를 모두 지운 빈 패턴을 저장할 수 있습니다
노트 최대 5000개로 제한	실수로 아주 큰 목록을 보내 DB 가 멈추는 것을 막습니다
패턴 삭제 전에 클립 사용 검사	클립이 가리키는 패턴을 지우면 그 클립이 빈 껍데기가 되므로, 쓰이는 패턴은 400으로 막고 쓰이지 않는 패턴만 지웁니다
패턴 목록 응답에 노트를 담지 않음	목록은 가볍게 두고, 노트는 상세 조회에서 받습니다 (FR-232, FR-233)
`note_id` 를 요청에서 받지 않음	통째 교체라 노트 번호를 서버가 새로 매깁니다

4. 검증

`tests/test_pattern_routes.py` 열두 건이 모두 통과합니다. `test_create_pattern_201` 은 만들기 201, `test_create_pattern_viewer_is_403` 는 VIEWER 거절을, `test_replace_notes` 는 노트 통째 교체 결과를 확인합니다. `test_replace_notes_validates_pitch` 는 음높이 범위 422, `test_viewer_cannot_save_notes` 는 노트 저장 403, `test_non_member_gets_404` 는 남의 패턴 404를 확인합니다. `test_delete_pattern_204` 는 쓰이지 않는 패턴 삭제, `test_delete_pattern_used_by_clip_is_400` 은 클립이 쓰는 패턴 삭제 거절을 확인합니다. 같은 규칙을 `tests/test_clip_model.py` 의 `test_pattern_used_by_clip_cannot_be_deleted` 가 DB 쪽에서도 봅니다. `pytest -q` 는 1083 passed, `ruff check .` 는 오류 없이 통과합니다.

5. 남은 일 (사용자)

확인할 것	내용
문서 검토	S1-10a 서비스와 라우터 문서(277 부터 281 까지)가 모두 끝난 뒤 한 번에 검토합니다
화면 확인	피아노롤에서 노트를 찍고 저장한 뒤, 타임라인에서 패턴을 지우는 동작을 시퀀서에서 확인합니다

백엔드 280 — 클립 · 연결 라우터 (api/v1/clips.py · api/v1/connections.py)

항목	내용
파일	`studionx/backend/app/api/v1/clips.py`, `studionx/backend/app/api/v1/connections.py`
작업	S1-10a 클립·연결·패턴 API: 타임라인 클립과 트랙 간 연결의 HTTP 라우터 (PRD 02 FR-2196, FR-2197)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	이전 279 패턴 서비스, 다음 281 패턴 라우터

1. 하는 일

클립과 연결의 HTTP 길을 두 파일에 나누어 둡니다. `app/api/v1/router.py` 에서 두 라우터를 접두사 없이 붙이므로, 화면은 `/api/v1/tracks/{id}/clips` 같은 경로로 부릅니다. 라우터는 요청 모양과 상태 코드만 정하고, 권한과 404 그리고 고리 검사는 서비스(277, 278)에 그대로 넘깁니다.

메서드와 경로	하는 일	상태 코드
`GET /tracks/{track_id}/clips`	트랙에 놓인 클립 목록을 start_tick 순으로 읽기	200
`POST /tracks/{track_id}/clips`	트랙에 클립 하나 놓기	201
`PATCH /clips/{clip_id}`	클립의 위치, 길이, 페이드, 워프 등 수정	200
`DELETE /clips/{clip_id}`	클립 삭제 (가리키던 패턴은 남습니다)	204
`GET /tracks/{track_id}/connection`	이 트랙에서 나가는 연결 읽기	200, 없으면 404
`PUT /tracks/{track_id}/connection`	연결을 보낸 값으로 통째로 맞추기	200
`DELETE /tracks/{track_id}/connection`	연결 끄기 (이미 없어도 통과합니다)	204

```
clips.py         GET, POST /tracks/{id}/clips — clip_service.list_clips, create_clip
                 PATCH, DELETE /clips/{id}    — clip_service.update_clip, delete_clip

connections.py   GET, PUT, DELETE /tracks/{id}/connection — connection_service
                 PUT 은 새로 만들거나 통째로 덮어쓰고, DELETE 는 연결을 끕니다
```

2. 코드

```
@router.post(
    "/tracks/{track_id}/clips",
    response_model=ClipResponse,
    status_code=status.HTTP_201_CREATED,
    responses=WRITE_RESPONSES,
)
def create_clip(
    track_id: Annotated[int, Path(ge=1)],
    req: ClipCreateRequest,
    db: Db,
    current_user: CurrentUser,
):
```

```
    """트랙에 클립을 놓는다 (FR-2196) — OWNER·EDITOR 만."""
    clip = clip_service.create_clip(db, current_user, track_id, req)
    return ClipResponse.model_validate(clip)


@router.delete(
    "/clips/{clip_id}",
    status_code=status.HTTP_204_NO_CONTENT,
    responses={
        401: {"model": ErrorResponse},
        403: {"model": ErrorResponse},
        **NOT_FOUND_404,
    },
)
def delete_clip(
    clip_id: Annotated[int, Path(ge=1)],
    db: Db,
    current_user: CurrentUser,
):
    """클립 삭제 (FR-2196) — OWNER·EDITOR 만."""
    clip_service.delete_clip(db, current_user, clip_id)
    return Response(status_code=status.HTTP_204_NO_CONTENT)
```

연결 쪽은 PUT 이 저장, DELETE 가 끄기를 담당합니다.

```
@router.put(
    "/tracks/{track_id}/connection",
    response_model=TrackConnectionResponse,
    responses=WRITE_RESPONSES,
)
def set_connection(
    track_id: Annotated[int, Path(ge=1)],
    req: TrackConnectionSetRequest,
    db: Db,
    current_user: CurrentUser,
):
    """연결을 보낸 값으로 통째로 맞춘다 (FR-2197) — OWNER·EDITOR 만."""
    connection = connection_service.set_connection(db, current_user, track_id, req)
    return TrackConnectionResponse.model_validate(connection)


@router.delete(
    "/tracks/{track_id}/connection",
    status_code=status.HTTP_204_NO_CONTENT,
    responses={
        401: {"model": ErrorResponse},
        403: {"model": ErrorResponse},
        **NOT_FOUND_404,
    },
)
def delete_connection(
    track_id: Annotated[int, Path(ge=1)],
    db: Db,
    current_user: CurrentUser,
):
    """연결 끄기 (FR-2197) — OWNER·EDITOR 만. 연결이 없어도 204."""
```

```
    connection_service.delete_connection(db, current_user, track_id)
    return Response(status_code=status.HTTP_204_NO_CONTENT)
```

문법 풀이

식	뜻
`Annotated[int, Path(ge=1)]`	경로의 번호는 1 이상만 받습니다. 0 이나 글자가 들어오면 422입니다
`status_code=status.HTTP_201_CREATED`	만들기 응답 상태 코드를 201 로 바꿉니다
`status.HTTP_204_NO_CONTENT`	지우기는 내용 없는 204 로 보냅니다
`response_model=ClipResponse`	서비스가 돌려준 행을 응답 스키마 칸으로만 바꿔 내보냅니다
`ClipResponse.model_validate(clip)`	DB 행을 응답 모델로 검증하면서 변환합니다
`responses=WRITE_RESPONSES`	문서에 400, 401, 403, 404 모양을 적어 둡니다
`**NOT_FOUND_404`	404 항목을 사전에 합쳐 넣습니다
`req: ClipCreateRequest`	본문을 검증해 잘못된 값은 422 로 막고 넘깁니다
`CurrentUser`, `Db`	`app/api/deps.py` 별칭입니다. 로그인 사용자와 DB 세션을 자동으로 받습니다

3. 왜 이렇게

선택	이유
두 라우터를 접두사 없이 붙임	클립은 `/tracks/{id}/clips` 와 `/clips/{id}` 로, 연결은 `/tracks/{id}/connection` 하나로 읽기 쉽습니다
라우터를 얇게 둠	권한, 404, 고리 검사는 서비스 한 곳에 있으므로 라우터는 부르기와 응답 변환만 합니다
연결 저장을 PUT 으로 둠	한 트랙에서 나가는 연결은 하나뿐이라, 새로 만들기와 고치기가 사실은 같은 동작입니다
연결 삭제를 204 로 두고 오류를 안 냄	이미 꺼진 연결을 다시 끌 때 화면이 오류로 멈추지 않습니다
클립 삭제 뒤에도 패턴을 남김	패턴은 다른 클립이 쓸 수 있기 때문입니다 (277 문서)
응답 스키마를 항상 지정	문서와 실제 응답 칸이 같아지고, 비밀 칸이 새는 것을 막습니다

4. 검증

`tests/test_clip_routes.py` 와 `tests/test_connection_routes.py` 가 이 두 파일을 직접 부릅니다. `test_clip_routes_registered` 와 `test_connection_routes_registered` 가 `app.openapi()["paths"]` 에 경로가 실렸는지 확인하고, `test_create_midi_clip_201`, `test_create_audio_clip_201`, `test_patch_clip`, `test_delete_clip` 은 201, 200, 204 상태 코드를 확인합니다. `test_put_creates_and_get_returns`, `test_put_replaces_existing`, `test_delete_turns_connection_off` 는 PUT 이 만들고 덮어쓰며 DELETE 가 끄는 동작을 확인합니다. `test_viewer_can_read_but_not_write` 와 `test_viewer_cannot_write` 는 권한, `test_needs_login` 은 로그인 없음 401 을 확인합니다. `pytest -q` 는 1083 passed, `ruff check .` 는 오류 없이 통과합니다.

5. 남은 일 (사용자)

확인할 것	내용
문서 검토	S1-10a 서비스와 라우터 문서(277 부터 281 까지)가 모두 끝난 뒤 한 번에 검토합니다

확인할 것	내용
화면 확인	타임라인에 클립을 놓고 옮기고 지우기, 연결 켜기와 끄기를 시퀀서에서 직접 확인합니다

백엔드 281 — 패턴 라우터 (api/v1/patterns.py)

항목	내용
파일	`studionx/backend/app/api/v1/patterns.py`
작업	S1-10a 클립·연결·패턴 API: 패턴과 노트 저장의 HTTP 라우터 (PRD 02 FR-2196, FR-231)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	이전 280 클립 · 연결 라우터, 다음 S1-10a 문서 정리

1. 하는 일

패턴 여섯 길을 한 파일에 둡니다. `app/api/v1/router.py` 에서 `/patterns` 접두사로 붙이므로, 화면은 `/api/v1/patterns` 아래의 경로로 부릅니다. 라우터는 요청 모양과 상태 코드만 정하고, 권한과 404 그리고 클립 사용 검사는 `pattern_service`(279)에 그대로 넘깁니다.

메서드와 경로	하는 일	상태 코드
`GET /patterns?project_id=`	프로젝트의 패턴 목록 (노트는 담지 않습니다)	200
`POST /patterns`	새 빈 패턴 만들기	201
`GET /patterns/{pattern_id}`	패턴 한 건과 노트 전부 (피아노롤이 열 때)	200
`PUT /patterns/{pattern_id}/notes`	노트를 보낸 목록으로 통째로 교체	200
`PATCH /patterns/{pattern_id}`	이름, 색, 길이, 스윙 수정	200
`DELETE /patterns/{pattern_id}`	패턴 삭제 (클립이 쓰면 400, 아니면 204)	204

```
POST   /patterns             PatternCreateRequest          → PatternResponse (201)
PUT    /patterns/{id}/notes  NotesSaveRequest {notes:[…]}  → PatternResponse (노트 포함)
DELETE /patterns/{id}        본문 없음                      → 204, 쓰이는 패턴이면 400
```

2. 코드

```
@router.post(
    "",
    response_model=PatternResponse,
    status_code=status.HTTP_201_CREATED,
    responses={
        400: {"model": ErrorResponse},
        403: {"model": ErrorResponse},
        404: {"model": ErrorResponse},
    },
)
def create_pattern(
    data: PatternCreateRequest,
    user: CurrentUser,
    db: Db,
) -> PatternResponse:
    """새 빈 패턴 만들기 (FR-2196) — OWNER·EDITOR 만."""
    return pattern_service.create_pattern(db, user, data)
```

```
@router.put(
    "/{pattern_id}/notes",
    response_model=PatternResponse,
    responses={403: {"model": ErrorResponse}, 404: {"model": ErrorResponse}},
)
def replace_notes(
    pattern_id: int,
    data: NotesSaveRequest,
    user: CurrentUser,
    db: Db,
) -> PatternResponse:
    """노트 통째 교체 (FR-231) — OWNER·EDITOR 만 (VIEWER 는 403)."""
    return pattern_service.replace_notes(db, user, pattern_id, data)


@router.delete(
    "/{pattern_id}",
    status_code=status.HTTP_204_NO_CONTENT,
    responses={
        400: {"model": ErrorResponse},
        403: {"model": ErrorResponse},
        404: {"model": ErrorResponse},
    },
)
def delete_pattern(
    pattern_id: Annotated[int, Path(ge=1)],
    user: CurrentUser,
    db: Db,
):
    """패턴 삭제 (FR-2196) — OWNER·EDITOR 만. 클립이 쓰고 있으면 400."""
    pattern_service.delete_pattern(db, user, pattern_id)
    return Response(status_code=status.HTTP_204_NO_CONTENT)
```

목록과 상세는 다음과 같이 서비스에 넘기고 응답 모델을 그대로 돌려줍니다.

```
@router.get("", response_model=PatternListResponse)
def list_patterns(
    user: CurrentUser,
    db: Db,
    project_id: Annotated[int, Query(ge=1)],
) -> PatternListResponse:
    """프로젝트의 패턴 목록 (FR-233) — 멤버면 누구나. 노트는 담지 않는다."""
    return pattern_service.list_patterns(db, user, project_id)


@router.get(
    "/{pattern_id}",
    response_model=PatternResponse,
    responses={404: {"model": ErrorResponse}},
)
def get_pattern(pattern_id: int, user: CurrentUser, db: Db) -> PatternResponse:
    """패턴 한 건과 노트 전부 (FR-232) — 피아노롤이 열 때 쓴다."""
    return pattern_service.get_pattern(db, user, pattern_id)
```

문법 풀이

식	뜻
`@router.post("", ...)`	접두사 `/patterns` 와 붙어 `POST /patterns` 가 됩니다. 빈 경로도 맞습니다
`Annotated[int, Query(ge=1)]`	쿼리 `project_id` 는 1 이상만 받습니다
`status_code=status.HTTP_201_CREATED`	만들기는 본문과 함께 201 로 보냅니다
`NotesSaveRequest`	`{ "notes": [ … ] }` 본문을 검증합니다. 노트 칸 범위가 어긋나면 422입니다
`responses={400: ..., 403: ..., 404: ...}`	문서에 잘못된 요청과 권한, 없음의 모양을 적어 둡니다
`status.HTTP_204_NO_CONTENT`	삭제는 내용 없는 204 로 보냅니다
`pattern_id: Annotated[int, Path(ge=1)]`	경로의 패턴 번호는 1 이상만 받습니다
`CurrentUser`, `Db`	로그인 사용자와 DB 세션을 자동으로 받는 별칭입니다

3. 왜 이렇게

선택	이유
여섯 길을 한 파일에 둠	패턴 하나가 주는 자원이라 `patterns` 아래에 모아 두면 문서와 화면에서 찾기 쉽습니다
노트 저장을 PUT 으로 둠	피아노롤은 지금 보이는 노트 전부를 다시 보내므로, 부분 고치기가 아니라 통째 교체가 맞습니다
만들기와 노트 저장에 403 응답을 표시	VIEWER 는 쓰기를 못 하므로 403이 실제로 날 수 있음을 문서에 적어 둡니다
삭제에 400 응답을 표시	클립이 쓰는 패턴은 지울 수 없으므로 400이 실제로 날 수 있음을 문서에 적어 둡니다
목록과 상세의 응답 모델을 따로 둠	목록은 노트 없이 가볍고, 상세는 노트를 담습니다 (279 문서)
라우터를 얇게 둠	권한, 404, 클립 사용 검사는 서비스 한 곳에 있으므로 라우터는 부르기만 합니다

4. 검증

`tests/test_pattern_routes.py` 가 이 파일을 직접 부릅니다. `test_pattern_routes_registered` 가 `app.openapi()["paths"]` 에 여섯 경로가 실렸는지 확인하고, `test_create_pattern_201` 은 만들기 201, `test_create_pattern_viewer_is_403` 는 VIEWER 403, `test_replace_notes` 와 `test_replace_notes_validates_pitch` 는 노트 교체와 범위 검사를, `test_delete_pattern_204` 와 `test_delete_pattern_used_by_clip_is_400` 은 삭제 204 와 클립 사용 400 을 확인합니다. `test_list_patterns`, `test_get_pattern_with_notes`, `test_patch_pattern`, `test_non_member_gets_404`, `test_viewer_cannot_save_notes` 도 모두 통과합니다. `pytest -q` 는 1083 passed, `ruff check .` 는 오류 없이 통과합니다.

5. 남은 일 (사용자)

확인할 것	내용
문서 검토	S1-10a 서비스와 라우터 문서(277 부터 281 까지)가 모두 끝난 뒤 한 번에 검토합니다
화면 확인	패턴 만들기, 피아노롤 노트 저장, 패턴 삭제를 시퀀서에서 직접 확인합니다